Seeing a GPU from the Outside: Counter-Free Performance Observability for Non-Privileged Tenants

Anonymous Authors

***Abstract*—Cloud GPU tenants increasingly run inside containers that cannot read hardware performance counters, lock clocks, or invoke privileged profiling interfaces. This paper studies what a tenant can still learn about its own kernel execution from permission-free signals. We present a counter-free observation pipeline built from dynamic binary instrumentation, unprivileged timing records, static binary inspection, and read-only device covariates. The pipeline classifies profiler metrics into recoverable quantities, which are reported only after passing predefined accuracy and robustness checks, and non-identifiable quantities, which receive falsifiable certificates naming the missing observation. The central finding is an observability boundary: traffic and cache behavior that re-enters the global-address stream can be reconstructed from user-space traces, while data reuse staged entirely in on-chip storage is erased from the observable surface. On two NVIDIA GPUs of different generations—a Blackwell-class RTX 5060 and an Ampere-class RTX 3090 Ti—the framework covers 127 correctness-passing native kernel ports and 178 completed kernel instances, respectively, produces measurement evidence for 222 and 225 kernels of the SOL-ExecBench suite, recovers L1→L2 read-sector counts with geometric mean relative error (GMRE) 2.7×10^{-12}, and keeps calibrated benchmark hit-rate error in the 0.03–0.06 range. The approach turns profiling under counter lockdown into an identifiability problem: the system reports estimates only when the permitted observations support them, and otherwise names the missing observation.**

***Index Terms*—GPU performance measurement, non-root profiling, hardware performance counters, cache modeling, identifiability, uncertainty quantification.**

I. Introduction

Performance optimization starts with measurement. On a GPU, the standard measurement instrument is the hardware performance counter: a privileged surface that reports cache hits, memory transactions, stall reasons, and other execution facts that are otherwise hidden inside the device. That assumption is fragile in the environment where many accelerator workloads now run. After the CVE-2018-6260 disclosure showed that shared counters can expose cross-tenant information [1], [2], NVIDIA drivers began restricting counter access to administrators by default. An ordinary tenant sees `ERR_NVGPUCTRPERM` unless the host administrator changes driver policy [3]. Recent toolkit releases also retire older counter paths [4]. In a rented container, the tenant usually owns neither the host nor the policy switch.

The result is a practical measurement wall. **Nsight Compute** (NCU), NVIDIA's per-kernel hardware profiler, and the counter-bearing interfaces of **CUPTI**, the CUDA Profiling Tools Interface, fail when the host keeps the default restriction in place [4], [5]; administrative clock controls and persistence controls are unavailable for the same reason. Yet a tenant still has legal observations: dynamic instrumentation of its own process, unprivileged activity records, CUDA events, static disassembly, and read-only status covariates. This paper asks what those signals determine.

Question. *Given only the signals a non-privileged process may legally observe, which GPU performance metrics can be recovered, and which are structurally lost?*

Answer. The answer is not a single surrogate profiler. It is an identifiability map. For each metric, the system first asks whether the metric is determined by the available observations. If it is, the metric is reported only after passing the required cross-checks and uncertainty tests. If it is not, the system emits a certificate: the physical mechanism that hides the metric and the additional non-privileged observation that would overturn the verdict.

This framing changes the role of a profiler substitute. Prior work offers cycle-level simulation, analytic or learned performance prediction, roofline analysis, and trace-driven cache modeling. Each is useful, but each gives up at least one constraint in the tenant setting: privileged validation data, configuration knowledge, clock control, or suite-average claims that do not say which metrics are knowable for a particular run. We instead make the non-privileged observation surface the premise.

Contributions.

1) We define a counter-free measurement method that uses only tenant-visible signals and separates recoverable estimates from non-identifiable metrics by a predefined set of checks.
2) We show that cache behavior has a structural boundary: reuse visible in the global-address stream can be modeled by trace replay, while reuse staged entirely on chip is erased from that stream.
3) We build a framework that applies the method at scale to a modern GPU kernel benchmark, with per-kernel evidence availability, local reference measurements used only for validation, and reproducible reporting.

II. Background and Related Work

GPU execution and memory primer. A GPU kernel executes as thousands of threads grouped into 32-thread *warps*; each thread occupies a *lane* of its warp and runs on one of the

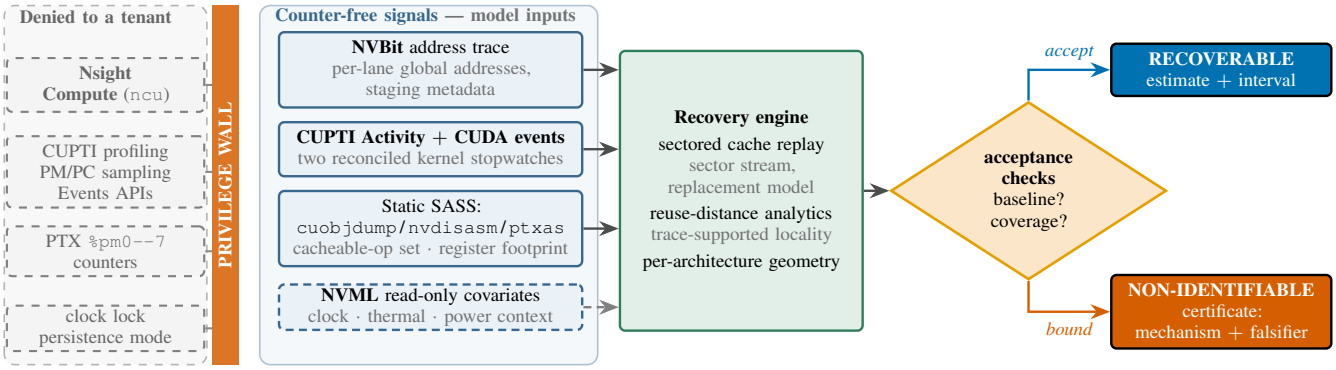


Fig. 1. **Counter-free observability under the privilege wall.** Counter-bearing profiler surfaces and clock controls are unavailable when the host keeps the post-CVE-2018-6260 default restriction in place. The pipeline uses only permission-free signals from the tenant process, accepts recoverable metrics only after predefined checks, and reports non-identifiable metrics as mechanism-level certificates rather than unsupported point predictions.

chip's *streaming multiprocessors* (SMs). Memory instructions target either *global memory* (device DRAM, addressable by all threads) or fast on-chip storage: registers and the software-managed *shared memory* (a scratchpad). Global accesses pass through a two-level cache hierarchy (L1 per SM, L2 shared), and NVIDIA caches move data in 32-byte *sectors*, four per 128-byte cache line; adjacent lane accesses are *coalesced* into the minimal set of sector requests. Profilers such as Nsight Compute report this machinery—hit rates, sector traffic, *occupancy* (the fraction of warp slots resident on each SM), and instructions per cycle (IPC)—by reading hardware performance counters, the privileged surface this paper does without.

The measurement contract. On a host that leaves `NVreg_RestrictProfilingToAdminUsers` enabled, an unprivileged process cannot use **Nsight Compute**, CUPTI profiling interfaces, performance-monitor (PM) or program-counter (PC) sampling, event and metric APIs, performance counters visible from PTX (NVIDIA's virtual instruction set), or administrative clock controls [3]–[5]. The documented remedies require root, a host-side driver option, or an administrative container capability; all are outside the tenant's authority. The remaining substrate is thinner but real: **NVBit** dynamic binary instrumentation, which rewrites the tenant's own GPU binaries at load time to insert measurement callbacks [6], [7]; unprivileged **CUPTI Activity** records, timeline entries for kernel launches and memory copies [8]; CUDA events, user-space GPU stopwatches; **Nsight Systems** timelines [9]; static disassembly of SASS, the GPU's native machine code [10]; correctness and memory tools such as **compute-sanitizer** [11]; and read-only **NVML** device-status covariates, the library behind `nvidia-smi` [12].

Simulator and model baselines. GPGPU-Sim, Accel-Sim, NVArchSim, and recent trace-driven core models reconstruct GPU behavior by modeling large parts of the device [13]–[16]. Analytical and learned predictors, including the memory-/compute-warp-parallelism (MWP/CWP) lineage, GPUMech, MDM, GCoM, and cross-architecture predictors, produce fast estimates but are calibrated against privileged or simulator-derived ground truth [17]–[22]. Roofline analysis supplies a counter-light performance bound by capping a kernel at peak compute and peak memory bandwidth, but hierarchical GPU roofline variants still depend on per-level traffic counts [23], [24]. These systems answer "what does a model predict?"; our question is "what does the tenant's observation surface determine?"

Trace-driven cache modeling. Reuse distance—the number of distinct data blocks touched between two consecutive uses of the same block—gives a classical bridge from address traces to cache behavior [25]. GPU cache studies extend that idea to GPU hierarchies and memory coalescing [26]–[29]. Hierarchical reuse distance (HRD) preserves locality across sector and coarser cache granularities [30]. Separately, SHALOM studies CPU multicore sharing locality; its taxonomy motivates our visible-stream labels for local, sequential-sharing, concurrent-sharing, and private/cold accesses [31]. The distinction here is not the use of a trace; it is that the trace is not assumed complete. Modern tensor-core and attention-class kernels often stage reuse through scratchpad, registers, or bulk-transfer machinery. When that reuse does not re-enter the global-address stream, no HRD-style trace replay can recover the missing hit behavior without an additional observation.

Comparison metrics. Table IV compares this paper's reported metrics against the closest timing-backed GPU models. Metric-error cells use GMRE, the geometric mean relative error over per-cell relative errors, when paired predictions and references are available; otherwise, the table keeps the baseline paper's reported unit and labels it in the cell. The timing row reports elapsed or derived tool run time rather than accuracy.

Instrumentation and microbenchmarking. NVBit, SASSI, and programmable assembly-level probes expose user-space instrumentation paths that do not require counters [6], [7], [32]. GPU microbenchmarking similarly uses self-timed experiments to infer hardware geometry [33]–[36]. We use these facts as the observation and parameter substrate, then ask which profiler metrics they identify for an actual kernel execution.

Table I summarizes which baseline families provide the tenant-visible evidence needed for run-specific measurement, uncertainty intervals, per-family decisions, and explicit non-identifiability certificates.

TABLE I
BASELINE CAPABILITIES UNDER THE TENANT-VISIBLE CONTRACT.

	measures the run	verified intervals	per-family verdicts	erasure certificates
Cycle-level simulation [13], [14]	×	×	×	×
Analytical/learned models [17]–[20], [22]	×	×	×	×
Roofline / speed-of-light [23]	×	×	×	×
Trace-driven cache models [26]–[30]	✓	×	×	×
Assembly-level probes [32]	✓	×	×	×
This work	✓	✓	✓	✓

Note. A check indicates that the capability is available under the tenant-visible observation surface without privileged counters. An erasure certificate explicitly reports why a metric is not identifiable.

III. COUNTER-FREE OBSERVATION

The pipeline in Fig. 1 has three rules. First, deployed features must come from surfaces available to an ordinary user. Second, every estimate must pass predefined acceptance checks before it is reported. Third, when the available signals do not identify a metric, the system reports the missing mechanism rather than filling the gap with a fitted correction.

A. Observation Surfaces

The model inputs comprise four signal families. The address trace uses **NVBit** to record global load and store addresses with active-lane masks (which lanes of each warp participate in the access) and instruction context. The timing path reconciles **CUPTI Activity** records with CUDA events and discards measurements whose independent stopwatches disagree. Static SASS inspection records cache-eligibility and resource properties without executing the kernel. Read-only **NVML** covariates record context such as clocks, temperature, and power, but the deployed estimators do not treat those covariates as privileged counters.

The harness enforces the contract directly. Counter-derived features are excluded, and clock-changing controls are disabled. The privileged profiler appears only as an offline reference source on local hosts where the experimenter controls the machine; it is never part of the tenant-visible deployment path.

B. Hierarchical Reuse-Distance Recovery

The richest counter-free signal is the memory-address stream. A cache access hits when the data was used recently enough; base-layer reuse distance formalizes this as the number of distinct sectors touched since the previous access to the same sector [25]. The recovery engine uses a sectorized HRD profile. Recorded lane addresses are first coalesced into hardware-sector units following GPU cache-trace practice [26], [27]. The sector stream is then lifted into coarser cache granularities and request-type channels following HRD [30]. A SHALOM-inspired locality tag records only the local, sequential-sharing, concurrent-sharing, or private/cold reuse visible in the global address stream before replay [31]. The cross-check compares the replayed cache state with an independent HRD profile. The replay is intentionally narrow: it models address-supported locality, not scheduling, timing, or hidden on-chip state.

C. Acceptance Checks and Certificates

A bare estimate is not enough. Each recoverable metric is compared with the strongest available baseline and, when calibrated intervals are meaningful, its coverage is checked on held-out evidence using conformal prediction, a distribution-free technique that calibrates interval widths so that a stated fraction of held-out truths falls inside them [37]. A failed baseline, coverage, or robustness check produces an explicitly unsupported estimate, not a softened pass.

For a non-identifiable quantity, the output is a certificate. The certificate contains the affected metric, the hiding mechanism, the bounded evidence in hand, and the non-privileged observation that would falsify the verdict. This borrows the language of structural identifiability [38] but applies it to performance observability: before estimating a metric, the system asks whether the available signals determine it at all.

IV. IDENTIFIABILITY MAP

The map applies these checks to the profiler quantities that performance analysts normally consult: traffic between cache levels, cache hit rates, memory-system traffic, bandwidth, cycles, occupancy, instruction throughput, and related derived quantities. The result is a boundary: metrics tied directly to the recorded global-address stream can enter the recoverable tier. Metrics governed by hidden controller behavior, scheduler state, dynamic frequency, or on-chip staging receive certificates.

Throughout the map, a metric receives one of three verdicts: *recoverable* (the estimate passes every predefined check), *outside support* (an estimator exists but fails an accuracy, coverage, or robustness check), or *non-identifiable* (the required information is structurally absent from the observation surface, so the metric receives a certificate). A *cell* denotes one kernel–input-shape configuration.

Essential NCU-Metric Observability Map

Metric family	Result under tenant-visible signals
L1→L2 read sectors	RTX 5060 recoverable, GMRE 2.7×10^{-12} over 39 held-out cells; RTX 3090 Ti remains outside support because p90 error reaches 0.51.
L1→L2 write sectors	Recoverable on both GPUs; RTX 5060 GMRE 5.1×10^{-7} with 90/95% coverage 35/39 and 36/39; RTX 3090 Ti has 19/20 exact cells.
L1 sector hit rate	Outside support: Blackwell intervals reach the width floor, and Ampere has GMRE 0.89 over 30 cells.
L2 sector hit rate	Family-dependent: streaming kernels have median error 0.04; staged-reuse families range from 0.13 to 0.92 because reuse stays on chip.
DRAM traffic and bandwidth	Not identifiable without a non-counter view of controller transactions, coalescing/compression outcomes, and clock-stable byte timing.
Cycles, IPC, occupancy, throughput	Not identifiable from address traces because the missing state is scheduler residency, dynamic frequency, or hidden memory-system traffic.

Note. GMRE is geometric mean relative error; p90 is the 90th-percentile relative error. An interval at the *width floor* is maximally wide and therefore uninformative. This compact map summarizes the eleven-metric result set while keeping the detailed state-of-the-art comparison in Table IV.

The recoverable tier is intentionally narrow. The read-sector path is exact on Blackwell, the write-sector path clears the checks on both GPU generations, and the L1 hit-rate estimator remains outside support when its uncertainty or its handling of bypass-class loads (loads that skip L1 entirely) exceeds the stated budget. This ordering matters: the same address trace can determine traffic counts while still lacking enough information to determine every hit rate.

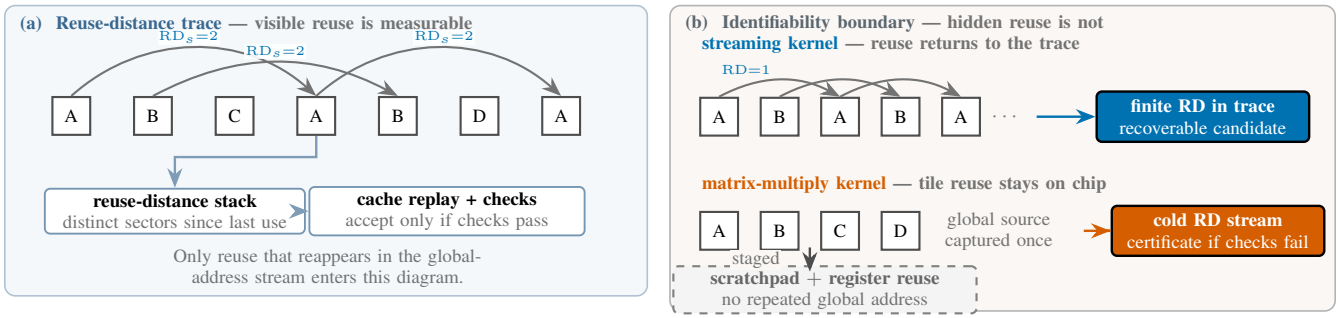


Fig. 2. **Why a reuse-distance trace succeeds and fails.** Visible reuse creates finite reuse distances that can be replayed. Reuse staged in scratchpad or registers can change hardware hit behavior without producing additional global-address observations, creating a structural erasure mechanism.

TABLE II
L2 HIT-RATE RECOVERABILITY BY KERNEL FAMILY.

Kernel family	n	median error	×threshold	Verdict: hidden mechanism
memory streaming	6	0.04	0.8×	**recoverable**: reuse re-traverses global memory
reduction / normalize	7	0.32	5.9×	**not identifiable**: cross-group partial-sum reuse unseen
intermediate buffers	7	0.13	2.4×	**not identifiable**: cache-resident staging buffers unseen
matrix multiply	9	0.92	16.7×	**not identifiable**: operand tiles staged on chip
register spill	0	—	—	**not identifiable**: no observed case to score; reuse is thread-private

Note. Errors are absolute hit-ratio fractions for the RTX 5060 L2 family study. The fixed acceptance threshold is 0.055. An independent checker agrees on all 29 scoreable cells; the suite spans 70 admitted kernel/shape cells across the two GPUs, and the RTX 3090 Ti confirms the same streaming-versus-staged boundary structurally.

Table II applies this boundary to the L2 hit ratio. Streaming kernels expose repeated sectors directly in the address stream and remain below the threshold. The families that dominate modern AI kernels do not: reductions and normalization reuse partial sums across cooperating groups, intermediate buffers can remain cache-resident between visible accesses, matrix multiply stages operand tiles through shared memory and registers after one global fetch, and register-spill reuse is thread-private. An HRD replay sees the global source traffic but not the staged reuse. A fitted correction can match a cohort only by importing assumptions not present in the observation surface; the map therefore treats such outcomes as certificates unless the additional staging observation is present and passes the same checks. For matrix multiply, for example, the visible trace records the global fetch of operand tiles, but the subsequent scratchpad destinations, register reuse, and on-chip residency do not generate new global-address observations. The staging metadata available in the current tenant trace identifies instruction context and global-source behavior; it does not yet expose enough destination or residency information to identify the L2 hits created by that staged reuse.

A. Validation Discipline

The validation suite is organized by metric family and by GPU generation. The two test devices are an RTX 5060, from NVIDIA's recent Blackwell generation (compute capability sm_120), and an RTX 3090 Ti, from the older Ampere generation (sm_86). The suite comprises five L2 kernel families, 70 admitted kernel/shape cells across the two GPUs, and separate held-out cohorts for traffic recovery. Each cell is scored against a local Nsight Compute reference measurement. For each metric, validation compares the counter-free estimate with the local privileged reference measurement and records whether the predefined checks pass. The reported metrics follow the conventions used by the simulation and analytical-modeling literature, but the decision rule is per metric and per family rather than a suite average. Correlation is treated as descriptive only: large dynamic ranges can make a failed estimator look aligned, so acceptance depends on metric-specific error, coverage, and robustness criteria.

The kernels are deliberately split into two groups. Synthetic microbenchmarks isolate one mechanism at a time: sector coalescing, read/write traffic, cache eligibility, hierarchical reuse distance, staged on-chip reuse, and stopwatch agreement. Native benchmark kernels then test whether the same evidence path survives real operator structure: tensor-core tiles, attention-style reuse, normalization and reduction, quantization, and fused movement. This separation keeps the validation falsifiable. A microbenchmark can diagnose which mechanism failed, while a native kernel tests whether that mechanism remains visible once scheduling, fusion, and library code are present.

The retained microbenchmark results show both sides of the decision rule. On the RTX 5060, L1→L2 read sectors are recovered exactly over 39 held-out kernel configurations, with GMRE 2.7×10^{-12} and no fitted parameters. L1→L2 write sectors are also accepted, with Blackwell GMRE 5.1×10^{-7} and verified 90/95% coverage of 35/39 and 36/39. On the RTX 3090 Ti, writes remain accepted: p90 error 0.00, 19/20 exact cells, mean absolute percentage error (MAPE) of 5%, and Pearson correlation $r>0.999$. The same cross-architecture pass does not hold for every metric. Ampere read sectors still show 11/13 exact cells, MAPE 19%, and $r=0.94$, but remain outside support because two Ampere L2 mechanisms—the persisting-window carveout, a region of L2 reserved for designated data, and full-line fetch granularity, which fetches the whole 128-byte line rather than individual sectors—push p90 error to 0.51 against the 0.25 target. L1 hit rate is outside the supported region on both GPUs: the Blackwell interval reaches

the width floor, and the Ampere closure has GMRE 0.89 over 30 cells. L2 hit ratio exposes the staged-reuse boundary directly: Blackwell has family median errors from 0.04 to 0.92 and only r=0.38 over 29 cells, while the Ampere fit reaches aggregate GMRE 0.064 but remains a fitted aggregate because leave-one-out and leave-one-kernel-out checks move it by 0.34 and 0.71. The Ampere result is therefore used as a structural confirmation of the hidden-reuse boundary, not as a promoted fitted measurement: the mechanism matches across generations, but the fitted number is too sensitive to the held-out kernel.

V. Framework at Benchmark Scale

The framework packages the method for an industrial GPU-kernel benchmark environment. The benchmark side supplies native CUDA/CUTLASS profiling targets for the workloads under study, numerical-correctness checks, and per-kernel metadata. The measurement side applies the permission-free signals from Section III-A to each kernel and records why evidence is absent: host unavailable, kernel failure, unsupported signal, or missing reference measurement. Missing data are therefore named explicitly rather than silently omitted.

The benchmark target is SOL-ExecBench, a speed-of-light suite that scores real-world GPU kernels against hardware limits [39]. Its 235 problems span four genres: two difficulty tiers that the suite names L1 (94 problems) and L2 (82)—a labeling unrelated to the L1/L2 caches—plus quantization (Quant, 33) and FlashInfer-derived (FIB, 26) kernels. The upstream references are not ideal profiling targets because Python dispatch, JIT compilation, and framework scheduling can share a timeline with the kernel. We therefore use native CUDA/CUTLASS ports for profiling, so both the counter-free surfaces and the local reference measurement attach to the same executable kernel body.

Local reference measurements. Nsight Compute is restricted to local machines where the experimenter controls the host. It supplies labels for validation and calibration only; it is not invoked on the target tenant path. This separation is essential: the deployed framework must run under the same non-privileged contract that motivates the paper.

Reporting rules. Tables and figures use fixed metric names, units, baselines, and acceptance rules. A metric is reported or marked outside support only under those stated rules. Table IV follows the same convention with GMRE used for metric-error cells.

The evaluated native benchmark set includes 127 correctness-passing Blackwell ports and 178 completed Ampere kernel instances. Evidence availability—the number of SOL-ExecBench kernels for which the counter-free pipeline produced usable signals—reaches 222 kernels on the RTX 5060 and 225 on the RTX 3090 Ti. The cache-prediction cohorts require both counter-free traces and repeatable local NCU reference measurements. On that subset, calibrated hit-rate prediction remains within the cohort thresholds: on the RTX 3090 Ti main set (n=34), leave-one-out mean absolute error is $0.033 \leq 0.05$ for L1 and $0.055 \leq 0.06$ for L2; on the RTX 5060 held-out cohort, the corresponding errors are 0.036 and 0.038. The stress cases (n=1 and n=2) remain within their looser threshold. These are benchmark-scale calibrated-prediction results, not per-kernel recovery verdicts: the L2 cohort is threshold-sensitive, and staged-reuse kernels still receive the family-level interpretation from Table II.

VI. Evaluation Protocol

This section consolidates the protocol behind the results of Sections IV and V and places them beside the closest prior models. Table III lists the evaluation questions and the evidence reported for each one. Each question has a fixed measurement, acceptance check, and failure reason. This prevents post-hoc reshaping of claims: a metric either passes the stated checks, remains outside support with the failed criterion named, or receives a non-identifiability certificate.

TABLE III
Evaluation questions and reported evidence.

Question	Reported evidence
Interconnect traffic	read/write-sector error, interval coverage, and support decision
L1 behavior	hit-rate error, interval width, and cache-eligibility check
L2 behavior	per-family error, support decision, and certificate when staged reuse is not visible
Benchmark scale	per-architecture coverage, named exclusions, and cohort-level decisions
Tenant portability	which permission-free signals are available when privileged profilers are excluded

Note. Each row states the evidence used to report an estimate, mark it outside support, or support a non-identifiability certificate.

TABLE IV
Metric-level comparison against timing-backed GPU models.

NCU metric(s)	This work error/runtime	Nugteren '14 error/runtime	Accel Sim'20 error/runtime
L1 sector hit rate (l1tex__t_sector_hit_rate.pct)	0.89 outside support	6.4/8.3 pp	0.76 NRMSE
L1 to L2 read sectors (l1tex__m_xbar2l1tex_read_sectors_mem_lg_op_ld.sum)	2.7×10^{-12}		0.03 NRMSE
L1 to L2 write sectors (l1tex__m_l1tex2xbar_write_sectors_mem_lg_op_st.sum)	5.1×10^{-7}		
L2 sector hit rate (calibrated cohorts) (lts__t_sector_hit_rate.pct)	**0.063** **0.069** cohort	0.117 0.414 GMRE	0.47 NRMSE
Tool runtime (elapsed time or derived run time)	60 s/kernel	cutcp: 10 s model + 240 s Ocelot trace	80 s/1M warp instr. 48 h trace gen.

Note. Metric rows report error against hardware or simulator ground truth for NCU metrics under the stated support or cohort scope; the final row reports exact elapsed or derived tool timings. GMRE is geometric mean relative error, computed over per-cell relative errors. When a baseline cannot be converted to GMRE from source-reported pairs, the table keeps the paper's original unit: pp denotes percentage-point hit-rate error and NRMSE denotes normalized root mean squared error. Blank cells indicate that the source does not report a timing-backed comparable metric. The Nugteren L1 pp values correspond to the 16KB and 48KB cache configurations reported in the source. The L2 row uses the calibrated-cache comparison cohorts shared with the HPCA'14 reuse-distance baseline; paired L2 values are for the Ampere (sm_86) then the Blackwell (sm_120) GPU, while Table II gives the family-level support boundary. Bold marks the lowest GMRE where units are directly comparable. Sources: Nugteren et al. [26] and Accel-Sim [14].

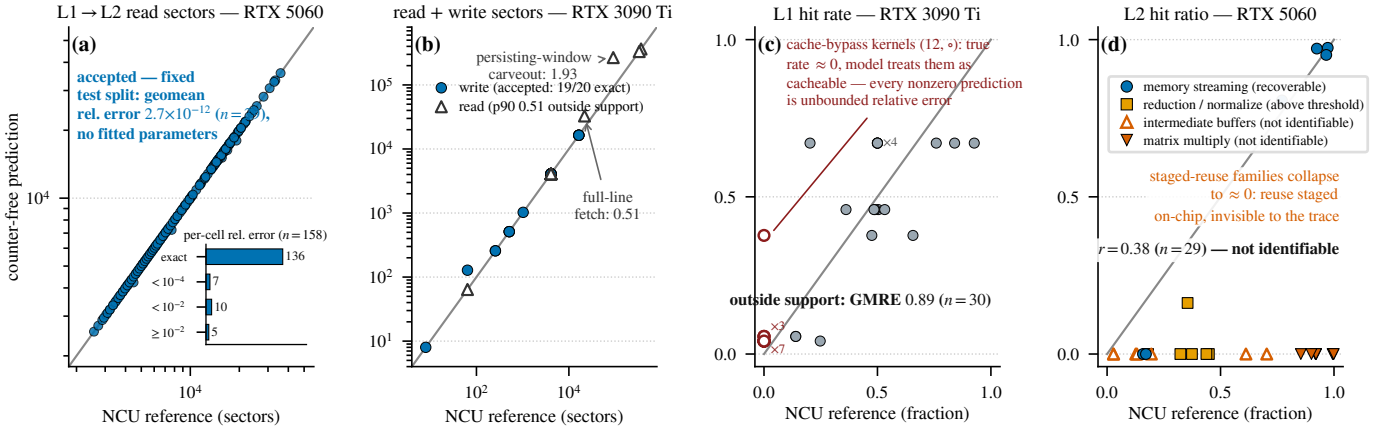


Fig. 3. **Microbenchmark validation against local Nsight Compute references.** Panels show Blackwell read-sector recovery, Ampere read/write traffic, Ampere L1 hit-rate boundary, and Blackwell L2 hit ratio by family. Accepted estimates follow the diagonal; unsupported estimates and certified families show the per-cell failure modes that suite averages would hide.

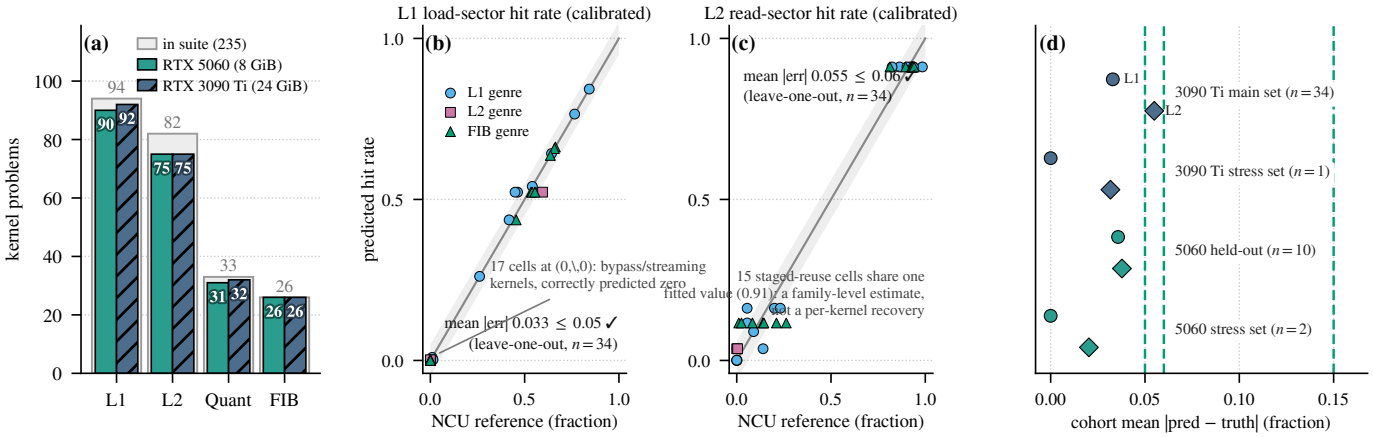


Fig. 4. **Benchmark-scale framework evidence.** Panel (a) reports SOL-ExecBench availability by genre on both GPUs. Panels (b)–(c) compare calibrated L1 and L2 hit-rate predictions with local Nsight Compute references on the RTX 3090 Ti cohort. Panel (d) summarizes cohort mean absolute errors against the thresholds shown in the panel; L1 points are circles and L2 points are diamonds.

The tables separate measured quantities from interpretation criteria. A metric is reported as recoverable only when it satisfies the stated error, coverage, and robustness checks. Otherwise, the evaluation names the reason the metric cannot be identified from tenant-visible signals.

VII. Discussion and Limitations

What the boundary means. The limit on counter-free observability is structural, not merely statistical. A failed cache estimate may indicate a modeling defect, but it may also indicate that the relevant information never entered the observation surface. The certificate form forces that distinction. It tells the practitioner whether a metric can be reported now, and it tells tool builders which additional non-privileged observation would make the metric testable.

Why bounded reporting matters. A profiler substitute that always returns a number is attractive but dangerous. It hides the difference between a recoverable counter, a calibrated cohort prior, and a structurally erased quantity. This work treats support boundaries as part of the measurement result. Marking an unsupported estimate preserves the validity of accepted claims and prevents suite-average accuracy from masking per-family failure.

Evidence discipline. Accuracy targets and workload partitions are fixed before scoring. Intervals are checked for coverage, recovered counts are cross-checked by independent calculations, and fitted corrections stay outside support when they fail holdout robustness. Several quantities therefore remain reported as unsupported even when a point estimate is numerically close; the paper treats those misses as results, not as text to be optimized away. The exact timings in Table IV are tool runtimes; architectural cycles, IPC, and throughput remain outside support when tenant-visible timers agree but the non-idle SM clock can still span 1688–2762 MHz.

External validity. The privileged reference requires local machines where the experimenter controls profiler permissions. A rented datacenter container can run the counter-free surfaces, but it cannot produce a privileged reference under the

same tenant contract. Datacenter deployment is therefore an availability and portability exercise; counter-level validation on those hosts requires administrator cooperation and is outside the tenant-only threat model.

VIII. Conclusion

This paper reframes GPU profiling under counter lockdown as an identifiability problem. Given only legal tenant-visible signals, some quantities can be recovered with predefined checks and uncertainty evidence; others are erased by hardware behavior that never reaches the global-address trace. The resulting profiler is not merely a counter surrogate. It is a boundary-drawing system: recover what the observation surface determines, mark what it does not, and name the extra observation required to change the verdict. The results show machine-precision L1→L2 read-sector recovery on Blackwell, coverage-checked write-sector recovery on both GPUs, streaming-kernel L2 hit-ratio recovery within threshold, and benchmark-scale availability of 222 and 225 SOL-ExecBench kernels on the two evaluated GPUs.

References

[1] H. Naghibijouybari, A. Neupane, Z. Qian, and N. Abu-Ghazaleh, "Rendered Insecure: GPU Side Channel Attacks are Practical," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2018, pp. 2139–2153.

[2] NVIDIA Corporation, "CVE-2018-6260: Risk of Application Data Leakage via GPU Performance Counters," MITRE CVE / NVIDIA Security Bulletin, 2018.

[3] ——, "ERR_NVGPUCTRPERM: Permission Issue with Performance Counters," https://developer.nvidia.com/ERR_NVGPUCTRPERM, 2019.

[4] ——, "CUPTI: CUDA Profiling Tools Interface," https://docs.nvidia.com/cupti/, 2024.

[5] ——, "Nsight Compute Profiling Guide," https://docs.nvidia.com/nsight-compute/, 2024.

[6] O. Villa, M. Stephenson, D. Nellans, and S. W. Keckler, "NVBit: A Dynamic Binary Instrumentation Framework for NVIDIA GPUs," in *Proc. 52nd Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO-52)*, 2019, pp. 372–383.

[7] M. Stephenson, S. K. S. Hari, Y. Lee, E. Ebrahimi, D. R. Johnson, D. Nellans, M. O'Connor, and S. W. Keckler, "Flexible Software Profiling of GPU Architectures," in *Proc. 42nd Annual Int. Symp. on Computer Architecture (ISCA)*, 2015, pp. 185–197.

[8] NVIDIA Corporation, "CUPTI Activity API," CUPTI User Guide, https://docs.nvidia.com/cupti/, 2024.

[9] ——, "Nsight Systems User Guide," https://docs.nvidia.com/nsight-systems/, 2024.

[10] ——, "CUDA Binary Utilities," https://docs.nvidia.com/cuda/cuda-binary-utilities/, 2024.

[11] ——, "Compute Sanitizer User Manual," https://docs.nvidia.com/compute-sanitizer/, 2024.

[12] ——, "NVML: NVIDIA Management Library Reference Manual," https://docs.nvidia.com/deploy/nvml-api/, 2024.

[13] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA Workloads Using a Detailed GPU Simulator," in *Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS)*, 2009, pp. 163–174.

[14] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling," in *Proc. ACM/IEEE 47th Annual Int. Symp. on Computer Architecture (ISCA)*, 2020, pp. 473–486.

[15] O. Villa, D. Lustig, Z. Yan, E. Bolotin, Y. Fu, N. Chatterjee, N. Jiang, and D. Nellans, "Need for Speed: Experiences Building a Trustworthy System-Level GPU Simulator," in *Proc. IEEE Int. Symp. on High-Performance Computer Architecture (HPCA)*, 2021, pp. 868–880.

[16] R. Huerta, M. Abaie Shoushtary, J.-L. Cruz, and A. González, "Dissecting and Modeling the Architecture of Modern GPU Cores," in *Proc. 58th Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, 2025.

[17] S. Hong and H. Kim, "An Analytical Model for a GPU Architecture with Memory-Level and Thread-Level Parallelism Awareness," in *Proc. 36th Annual Int. Symp. on Computer Architecture (ISCA)*, 2009, pp. 152–163.

[18] J.-C. Huang, J. H. Lee, H. Kim, and H.-H. S. Lee, "GPUMech: GPU Performance Modeling Technique Based on Interval Analysis," in *Proc. 47th Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, 2014, pp. 268–279.

[19] L. Wang, M. Jahre, A. Adileh, and L. Eeckhout, "MDM: The GPU Memory Divergence Model," in *Proc. 53rd Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, 2020, pp. 1009–1021.

[20] J. Lee, Y. Ha, S. Lee, J. Woo, J. Lee, H. Jang, and Y. Kim, "GCoM: A Detailed GPU Core Model for Accurate Analytical Modeling of Modern GPUs," in *Proc. ACM/IEEE 49th Annual Int. Symp. on Computer Architecture (ISCA)*, 2022, pp. 424–436.

[21] N. Ardalani, C. Lestourgeon, K. Sankaralingam, and X. Zhu, "Cross-Architecture Performance Prediction (XAPP) Using CPU Code to Predict GPU Performance," in *Proc. 48th Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, 2015, pp. 725–737.

[22] S. Lee, A. Phanishayee, and D. Mahajan, "Forecasting GPU Performance for Deep Learning Training and Inference," in *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[23] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[24] C. Yang, T. Kurth, and S. Williams, "Hierarchical Roofline Analysis for GPUs: Accelerating Performance Optimization for the NERSC-9 Perlmutter System," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 20, p. e5547, 2020.

[25] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, "Evaluation Techniques for Storage Hierarchies," *IBM Systems Journal*, vol. 9, no. 2, pp. 78–117, 1970.

[26] C. Nugteren, G.-J. van den Braak, H. Corporaal, and H. E. Bal, "A Detailed GPU Cache Model Based on Reuse Distance Theory," in *Proc. 20th IEEE Int. Symp. on High-Performance Computer Architecture (HPCA)*, 2014, pp. 37–48.

[27] M. Kiani and A. Rajabzadeh, "Efficient Cache Performance Modeling in GPUs Using Reuse Distance Analysis," *ACM Transactions on Architecture and Code Optimization*, vol. 15, no. 4, pp. 58:1–58:24, 2018.

[28] Y. Arafa, A.-H. A. Badawy, G. Chennupati, A. Barai, N. Santhi, and S. Eidenbenz, "Fast, Accurate, and Scalable Memory Modeling of GPGPUs Using Reuse Profiles," in *Proc. 34th ACM Int. Conf. on Supercomputing (ICS)*, 2020.

[29] Y. Arafa, A.-H. A. Badawy, A. ElWazir, A. Barai, A. Eker, G. Chennupati, N. Santhi, and S. Eidenbenz, "Hybrid, Scalable, Trace-Driven Performance Modeling of GPGPUs," in *Proc. Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[30] R. K. V. Maeda, Q. Cai, J. Xu, Z. Wang, and Z. Tian, "Fast and Accurate Exploration of Multi-Level Caches Using Hierarchical Reuse Distance," in *Proc. 23rd IEEE Int. Symp. on High-Performance Computer Architecture (HPCA)*, 2017, pp. 145–156.

[31] F. Jiang, R. K. V. Maeda, J. Feng, S. Chen, L. Chen, X. Li, and J. Xu, "Fast and Accurate Statistical Simulation of Shared-Memory Applications on Multicore Systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 10, pp. 2455–2469, 2022.

[32] S. Huang and C. Wu, "Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing," in *Proc. 19th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2025.

[33] H. Wong, M.-M. Papadopoulou, M. Sadooghi-Alvandi, and A. Moshovos, "Demystifying GPU Microarchitecture through Microbenchmarking," in *Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS)*, 2010, pp. 235–246.

[34] X. Mei and X. Chu, "Dissecting GPU Memory Hierarchy Through Microbenchmarking," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 1, pp. 72–86, 2017.

[35] Z. Jia, M. Maggioni, B. Staiger, and D. P. Scarpazza, "Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking," arXiv preprint arXiv:1804.06826, 2018.

[36] A. Jarmusch, N. Graddon, and S. Chandrasekaran, "Dissecting the NVIDIA Blackwell Architecture with Microbenchmarks," arXiv preprint arXiv:2507.10789, 2025.

[37] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.

[38] A. F. Villaverde, A. Barreiro, and A. Papachristodoulou, "Structural Identifiability of Dynamic Systems Biology Models," *PLOS Computational Biology*, vol. 12, no. 10, p. e1005153, 2016.

[39] E. Lin *et al.*, "SOL-ExecBench: Speed-of-Light Benchmarking for Real-World GPU Kernels Against Hardware Limits," arXiv preprint arXiv:2603.19173, 2026.